Dog Breed Identification using Transfer Learning

MILESTONE 1: IDEATION PHASE

The project Dog Breed Identification using Transfer Learning focuses on developing an intelligent system that can automatically identify the breed of a dog from an uploaded image. Many people are unable to recognize dog breeds correctly due to similarities between breeds and lack of expert knowledge. This creates problems in real-world situations such as pet adoption platforms, lost dog identification, and veterinary clinics.

To solve this problem, the project uses Transfer Learning, which allows the system to reuse knowledge from a pre-trained deep learning model instead of training a model from scratch. A pre-trained Convolutional Neural Network such as VGG19 is used to extract important visual features from dog images. These features are then used to classify the image into one of the predefined dog breed categories.

The system allows users to upload a dog image through a simple web interface. The uploaded image is preprocessed and passed to the trained model, which predicts the most probable dog breed. The predicted breed is then displayed to the user clearly on the screen.

This approach improves accuracy, reduces training time, and works efficiently even with limited training data. The project makes dog breed identification easy, fast, and accessible for everyone without requiring expert knowledge.

Problem Statement:

Identifying dog breeds accurately from images is a challenging task due to factors such as variations in lighting, pose, background, and similarities between different breeds. Traditional image classification methods require large labelled datasets and extensive computational resources, which are often not available.

In real-world scenarios like pet adoption platforms, lost dog identification, and veterinary diagnosis, incorrect or unknown breed identification can lead to poor decision-making, delayed assistance, or lack of proper care.

Therefore, there is a need for an efficient, accurate, and user-friendly system that can automatically identify dog breeds from images using advanced deep learning techniques, even when training data and resources are limited.

This project addresses the problem by using Transfer Learning, which leverages pre-trained convolutional neural networks to achieve high accuracy with less training time and computational cost.

For example, when a user visits an online pet adoption platform, they may want to search for dogs of a specific breed, but the breed information is often missing or inaccurate. Similarly, when a user finds a lost dog, they may not know the dog's breed, which makes it difficult to create an accurate description for helping reunite the dog with its owner. In veterinary clinics, users such as veterinarians and pet owners may not be sure about the breed of a dog, which is important for understanding health risks, behaviour, and treatment options.

Currently, there is no simple and user-friendly system that allows users to upload a dog image and instantly get an accurate breed prediction. Manual identification requires expert knowledge and is time-consuming, while traditional image classification systems require large datasets and high computational resources.

Therefore, users need a reliable, fast, and easy-to-use application that can automatically identify dog breeds from images using advanced machine learning techniques. The Dog Breed Identification using Transfer Learning project solves this problem by allowing users to upload a dog image and receive an accurate breed prediction, making breed identification accessible to everyone in real-world scenarios.

Scenarios:

Pet Adoption Platform:

Scenario: An online pet adoption platform wants to improve the user experience by automatically categorizing the dog breeds of animals available for adoption based on uploaded images. This helps potential adopters quickly find dogs that match their preferences.

Lost Dog Identification:

Scenario: A person finds a lost dog and wants to help reunite it with its owner. However, they are unsure of the dog's breed, making it challenging to create an accurate description for missing pet posters or online announcements.

Veterinary Diagnosis Support:

Scenario: A veterinarian needs assistance in identifying the breed of a mixed-breed dog brought in for a health checkup. Knowing the breed can provide valuable insights into potential genetic predispositions to certain health conditions or behavioral traits.

Technologies Used:

1. Python

- Core programming language used for model development and backend logic
- Provides strong support for machine learning and web frameworks

2. TensorFlow

- It is used to build, train, fine-tune, and load the CNN model
- Supports transfer learning with pre-trained models

3. Kera's

- High-level API of TensorFlow and Simplifies model building and training
- Used for loading pre-trained CNN architectures

4. NumPy

- Used for numerical computations
- Handles image arrays and model input processing

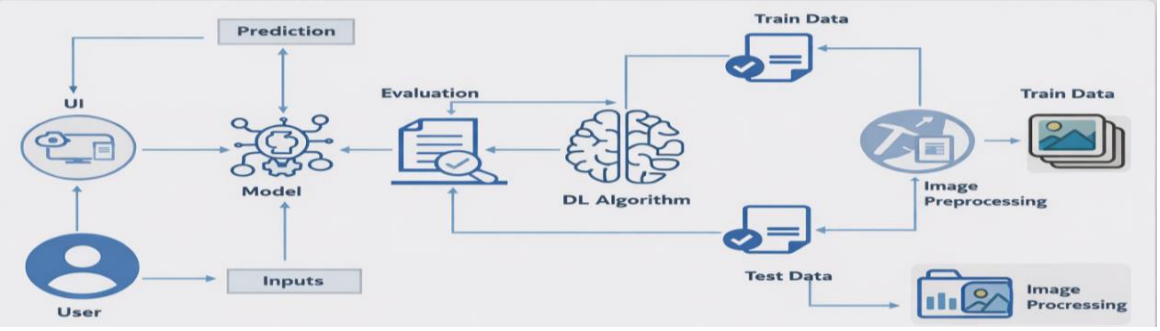
5. Flask

- Connects the trained model with the frontend
- Handles image uploads and prediction request

MILESTONE 2: REQUIREMENT ANALYSIS PHASE

Technical Architecture:

The Technical Architecture represents how data moves through the Dog Breed Identification system from the user to the final output.



The Dog Breed Identification system is built using a transfer learning–based architecture to efficiently classify dog breeds from images. The process starts when a user uploads a dog image through the web-based User Interface (UI). The input image is then passed to the image preprocessing module, where operations such as resizing, normalization, and scaling are performed to make the image compatible with the pre-trained model. The dataset is divided into training data and testing data. In this architecture, a pre-trained Convolutional Neural Network (CNN) model such as VGG19 is used as a feature extractor. Instead of training the entire network from scratch, the lower layers of the pre-trained model retain previously learned visual features, while the top layers are fine-tuned using the dog breed dataset. During the training phase, the model learns to map extracted features to specific dog breed classes. The trained model is then evaluated using test data to measure accuracy and performance. Once the evaluation is satisfactory, the final model is deployed. When a user uploads a new image, the trained transfer learning model predicts the most probable dog breed and displays the result through the UI. This architecture reduces training time, improves accuracy, and efficiently utilizes limited computational resources by leveraging knowledge from pre-trained models.

Prerequisites:

Anaconda Navigator is a free and open-source distribution of the Python and R programming languages for data science and machine learning related applications. It can be installed on Windows. Conda is an open-source, cross-platform, package management system. Anaconda comes with very nice tools like Jupyter Lab, Jupyter Notebook, QtConsole, Spyder, Rstudio, VS Code. For this project, we will be using Jupyter notebook and VS code.

Numpy

It is an open-source numerical Python library. It contains a multidimensional array and matrix data structures and can be used to perform mathematical operations

Scikit-learn

It is a free machine learning library for Python. It features various algorithms like support vector machine, random forests, and k-neighbors, and it also supports Python numerical and scientific libraries like NumPy and SciPy

Flask

Web framework used for building Web applications

Python packages

- open anaconda prompt as administrator
- Type “pip install NumPy” and click enter.
- Type “pip install pandas” and click enter.
- Type “pip install scikit-learn” and click enter.
- Type “pip install tensorflow==2.12.0” and click enter.
- Type “pip install keras==2.12.0” and click enter.
- Type “pip install Flask” and click enter.

Deep Learning Concepts

CNN: a convolutional neural network is a class of deep neural networks, most commonly applied to analysing visual imagery.

VGG19: VGG19 is a deep convolutional neural network architecture for image classification, consisting of 19 layers with small convolution filters.

Flask: Flask is a popular Python web framework, meaning it is a third-party Python library used for developing web applications.

Project Objectives:

By the end of this project I will:

- Know fundamental concepts and techniques of Convolutional Neural Network.
- Gain a broad understanding of image data.
- Know how to pre-process/clean the data using different data preprocessing techniques.
- know how to build a web application using the Flask framework.

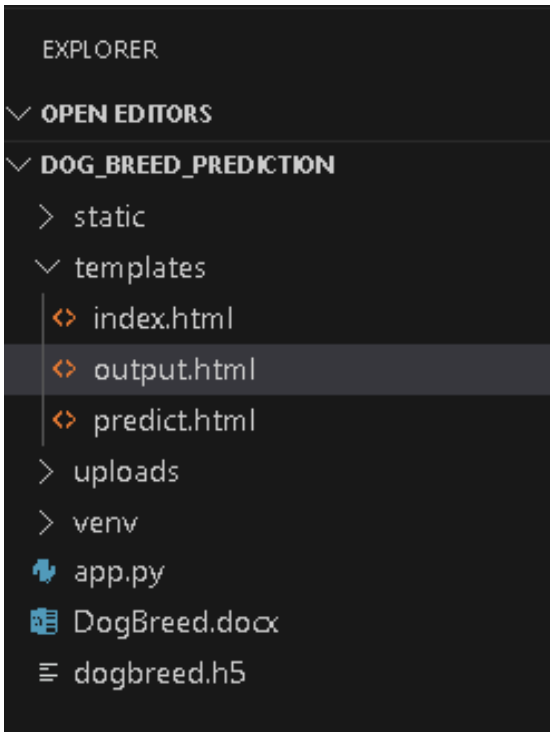
Project Flow:

- The user interacts with the UI (User Interface) to choose the image.
- The chosen image is analysed by the model which is integrated with flask application.
- CNN Models analyse the image, then prediction is showcased on Flask UI. To accomplish this, we must complete all the activities listed below.
- Data Collection.
    - Create Train and Test Folders.
- Data Preprocessing.
    - Import the Image Data Generator library.
    - Configure Image Data Generator class.
    - Apply Image Data Generator functionality to Train dataset and Test dataset
- Model Building
    - Import the model building Libraries.
    - Importing the VGG19.
    - Initializing the model.
    - Adding Fully connected Layer.
    - Configure the Learning Process.
    - Training and Testing the model.
    - Save the Model.
- Application Building
    - Create an HTML file.
    - Build Python Code.

MILESTONE 3: PROJECT DESIGN PHASE

Project Structure:

Create the Project folder which contains files as shown below.



- We are building a Flask Application that needs HTML pages stored in the templates.

  folder and a python script app.py for server-side scripting

- we need the model which is saved and the saved model in this content is a dogbreed.h5
- templates folder contains index.html, predict.html & output.html pages.
- The Static folder contains CSS file, images.

Problem Solution:

- The main problem addressed in this project is the automatic identification of dog breeds from images. Identifying dog breeds manually is difficult for normal users because many breeds look similar, and image conditions such as lighting, background, angle, and posture vary widely. Traditional image classification approaches require large labelled datasets and high computational resources, which are not always available.
- To overcome these challenges, this project uses a **Transfer Learning–based Deep Learning approach**. Transfer Learning allows the use of knowledge gained from a pre-trained model and applies it to a new but related task. This method significantly reduces training time, computational cost, and the need for large datasets while still achieving high accuracy.
- By using a pre-trained Convolutional Neural Network, the system can automatically extract important visual features such as shape, texture, colour, and patterns from dog images and accurately classify them into different breed categories.

Selected Algorithm

- The algorithm selected for this project is **VGG19 using Transfer Learning**.
- **VGG19** is a deep Convolutional Neural Network architecture that consists of 19 layers, including convolutional layers, pooling layers, and fully connected layers. It is pre-trained on the **ImageNet dataset**, which contains millions of images across 1000 different classes.

Proposed Solution:

- The proposed solution is a **Transfer Learning–based Dog Breed Identification System** designed using the VGG19 model and a web-based interface.

System Design Description

- The user uploads a dog image through a web application.
- The uploaded image is resized and normalized to match the input requirements of the VGG19 model.
- The preprocessed image is passed through the frozen layers of the VGG19 model to extract meaningful features.
- A Flatten layer converts extracted features into a one-dimensional vector.
- Fully connected Dense layers are added to classify the image into **20 different dog breeds**.
- The final output layer uses a **softmax activation function** to compute the probability of each breed.
- The breed with the highest probability is selected as the final prediction.
- The predicted dog breed is displayed to the user through the web interface.

MILESTONE 4: PROJECT PLANNING PHASE

I planned the complete workflow of the Dog Breed Identification project. Project planning helps in organizing tasks, allocating time, and ensuring smooth execution of the project within the given deadline.

Task Planning

The project was divided into the following tasks:

- Dataset collection from Kaggle
- Data preprocessing and augmentation
- Model building using transfer learning (VGG19)
- Model training and evaluation
- Saving the trained model
- Flask application development
- Testing and result verification

MILESTONE 5: PROJECT DEVELOPMENT PHASE

Data collection and Preprocessing

Activity 1: Dataset Collection

I started my project by collecting the required dataset for Dog Breed Identification. I downloaded the dataset from the Kaggle platform, specifically from the "Dog Breed Identification" competition. This dataset contains a large number of labelled images of dogs belonging to different breeds.

Link: https://www.kaggle.com/competitions/dog-breed-identification/data?select=train

In my project, I selected **20 different dog breeds** such as affenpinscher, beagle, appenzeller, basset, bluetick, boxer, cairn, doberman, german shepherd, golden retriever, kelpie, komondor, leonberg, mexican hairless, pug, redbone, shih-tzu, toy poodle, vizsla, and whippet. Each image in the dataset is associated with a unique image ID, which helps in identifying and managing the images properly.

After downloading the dataset, I stored all the images locally in my system so that they could be used for further preprocessing and training.

test	16-02-2026 14:07	File folder	
subset	16-02-2026 11:57	File folder	
train	16-02-2026 11:22	File folder	
Earlier this month			
labels	14-02-2026 12:15	CSV File	471 KB
test	14-02-2026 12:15	Compressed (zipp...	3,54,236 KB
train	14-02-2026 12:15	Compressed (zipp...	3,52,765 KB

Outcome:

At the end of this activity, I successfully collected a complete dataset containing images of 20 different dog breeds.

Activity 2: Organizing Images into Different Classes

After collecting the dataset, I organized the images into separate subdirectories based on their respective dog breed names. This step is very important because deep learning models require the dataset to be arranged in a structured, class-wise format.

Each image in the dataset has a unique image ID. For every image ID belonging to a particular breed, I created a source path by combining the dataset directory and the image filename in the format image_id.jpg. Then, I created a destination path by combining the corresponding breed folder and the same image filename.

Using the shutil.copyfile() function in Python, I copied each image from the source directory into its respective breed folder. I repeated this process for all images in the dataset so that each breed folder contained only images of that specific breed.

```
labels = pd.read_csv(LABELS_PATH)

SUBSET_TRAIN = os.path.join(BASE_PATH, "subset", "train")
os.makedirs(SUBSET_TRAIN, exist_ok=True)

breeds = labels['breed'].unique()[:20]

for breed in breeds:
    breed_folder = os.path.join(SUBSET_TRAIN, breed)
    os.makedirs(breed_folder, exist_ok=True)

    image_ids = labels[labels['breed'] == breed]['id']

    for img_id in image_ids:
        src = os.path.join(TRAIN_DIR, img_id + ".jpg")
        dst = os.path.join(breed_folder, img_id + ".jpg")

        if os.path.exists(src):
            shutil.copy(src, dst)

print("Subset created successfully")
```

```
Subset created successfully
```

Outcome:

By the end of this activity, all images were successfully organized into 20 separate class-wise folders, making the dataset ready for model training.

Activity 3: Image Preprocessing

Once the images were organized, I performed image preprocessing to improve the quality of the image data. Image preprocessing helps in removing unwanted distortions and enhancing important features required for training the model.

During this process, I applied several geometric transformations such as rotation, scaling, translation, and resizing. These transformations help the model learn better by making it robust to variations in image orientation, size, and lighting conditions.

Outcome:

After image preprocessing, the dataset became more suitable and efficient for training the deep learning model.

Activity 4: Importing ImageDataGenerator Library

To perform image data augmentation, I imported the ImageDataGenerator class from the TensorFlow Keras library. This class provides built-in support for generating new image samples by applying random transformations to the existing images during training.

Image data augmentation is useful when the dataset size is limited, as it helps increase the effective size of the training dataset.

```
from tensorflow.keras.preprocessing.image import ImageDataGenerator
print("ImageDataGenerator working")
```

```
ImageDataGenerator working
```

Outcome:

After importing ImageDataGenerator, my project environment was ready to perform image augmentation.

Activity 5: Configuring the ImageDataGenerator

Next, I configured the ImageDataGenerator by specifying various augmentation parameters. These parameters define how the images are modified during training.

I used techniques such as width and height shifting, horizontal and vertical flipping, image rotation, brightness adjustment, and zooming. These transformations are applied randomly to the images during training, allowing the model to learn from different variations of the same image.

```python
print(os.listdir(r"C:\Users\DELL\Downloads\DOG_BREED_PREDICTION\subset\train"))
```

Python

```
['african_hunting_dog', 'basenji', 'bedlington_terrier', 'bluetick', 'borzoi', 'boston_bull', 'boxer', 'd
```

```python
from tensorflow.keras.preprocessing.image import ImageDataGenerator

train_datagen = ImageDataGenerator(
    rescale=1./255,
    rotation_range=20,
    width_shift_range=0.2,
    height_shift_range=0.2,
    zoom_range=0.2,
    horizontal_flip=True
)
```

Outcome:

At the end of this activity, the ImageDataGenerator was fully configured for training and testing the dataset.

Activity 6: Applying ImageDataGenerator to Training and Testing Sets

After configuring the ImageDataGenerator, I applied it to both the training and testing datasets using the flow_from_directory() method. This method reads images directly from the class-wise folder structure and automatically assigns labels based on folder names.

I generated images in batches of size 32 and resized them to a fixed target size. The labels were encoded using categorical encoding, which is suitable for multi-class classification. During this step, I observed that **1696 images belonging to 20 classes** were used for training.

```
)

train_generator = train_datagen.flow_from_directory(
    r"C:\Users\DELL\Downloads\DOG_BREED_PREDICTION\subset\train",
    target_size=(224, 224),
    batch_size=32,
    class_mode='categorical'
)
```

```
Found 1696 images belonging to 20 classes.
```

Outcome:

The training and testing datasets were successfully prepared and ready for model building.

Model Building

Activity 1: Importing Model Building Libraries

In this activity, I imported all the required libraries such as TensorFlow, Keras, NumPy, and Matplotlib. These libraries are essential for building, training, and evaluating the Convolutional Neural Network model.

```
import os
import shutil
import numpy as np
import pandas as pd
import tensorflow as tf
```

Outcome:

The development environment was successfully prepared for model building.

Activity 2: Importing the VGG19 Model

I used the VGG19 model as the base model for my project. The VGG19 model is pre-trained on the ImageNet dataset, which contains millions of images across 1000 categories. Using a pre-trained model helps improve accuracy and reduces training time.

```
from tensorflow.keras.preprocessing.image import ImageDataGenerator
from tensorflow.keras.applications.vgg19 import VGG19, preprocess_input
from tensorflow.keras.layers import Dense, Flatten
from tensorflow.keras.models import Model
from tensorflow.keras.optimizers import Adam
```

```
BASE_PATH = r"C:\Users\DELL\Downloads\DOG_BREED_PREDICTION"
TRAIN_DIR = os.path.join(BASE_PATH, "train")
LABELS_PATH = os.path.join(BASE_PATH, "labels.csv")
```

Outcome:

The VGG19 pre-trained model was successfully loaded.

Activity 3: Initializing the Model

I initialized the VGG19 model by excluding its final fully connected layers. I froze all the layers of the pre-trained model by setting layer.trainable = False so that the learned weights would not be updated during training.

I then added a Flatten layer to convert the extracted features into a one-dimensional vector.

```
IMAGE_SIZE = (224, 224)
BATCH_SIZE = 32

train_datagen = ImageDataGenerator(
    preprocessing_function=preprocess_input,
    validation_split=0.2
)

train_generator = train_datagen.flow_from_directory(
    SUBSET_TRAIN,
    target_size=IMAGE_SIZE,
    batch_size=BATCH_SIZE,
    class_mode='categorical',
    subset='training'
)

val_generator = train_datagen.flow_from_directory(
    SUBSET_TRAIN,
    target_size=IMAGE_SIZE,
    batch_size=BATCH_SIZE,
    class_mode='categorical',
    subset='validation'
)

NUM_CLASSES = train_generator.num_classes
print("Classes:", NUM_CLASSES)
```

```
Found 1364 images belonging to 20 classes.
Found 332 images belonging to 20 classes.
Classes: 20
```

```
vgg = VGG19(
    input_shape=IMAGE_SIZE + (3,),
    weights='imagenet',
    include_top=False
)

for layer in vgg.layers:
    layer.trainable = False
```

Outcome:

The base model was initialized and ready for customization.

Activity 4: Adding Fully Connected Layers

After initializing the base model, I added fully connected Dense layers to the network. I specified the input shape as (128, 128, 3). The final Dense layer used the softmax activation function to classify the images into 20 different dog breed classes.

I also used the model.summary() function to view the complete architecture of the model.

```
x = Flatten()(vgg.output)
output = Dense(NUM_CLASSES, activation='softmax')(x)

model = Model(inputs=vgg.input, outputs=output)
model.summary()
```

Model: "functional"

Layer (type)	Output Shape	Param #
input_layer (InputLayer)	(None, 224, 224, 3)	0
block1_conv1 (Conv2D)	(None, 224, 224, 64)	1,792
block1_conv2 (Conv2D)	(None, 224, 224, 64)	36,928
block1_pool (MaxPooling2D)	(None, 112, 112, 64)	0
block2_conv1 (Conv2D)	(None, 112, 112, 128)	73,856
block2_conv2 (Conv2D)	(None, 112, 112, 128)	147,584
block2_pool (MaxPooling2D)	(None, 56, 56, 128)	0
block3_conv1 (Conv2D)	(None, 56, 56, 256)	295,168
block3_conv2 (Conv2D)	(None, 56, 56, 256)	590,080
block3_conv3 (Conv2D)	(None, 56, 56, 256)	590,080
block3_conv4 (Conv2D)	(None, 56, 56, 256)	590,080
block3_pool (MaxPooling2D)	(None, 28, 28, 256)	0
block4_conv1 (Conv2D)	(None, 28, 28, 512)	1,180,160

Outcome:

The CNN model architecture was successfully completed.

Activity 5: Configuring the Learning Process

I compiled the model using the Adam optimizer and categorical crossentropy loss function. I used accuracy as the evaluation metric to measure the model's performance.

```
model.compile(
    loss='categorical_crossentropy',
    optimizer=Adam(learning_rate=0.0001),
    metrics=['accuracy']
)
```

Outcome:

The model was successfully configured and ready for training.

Activity 6: Training the Model

I trained the model using the training dataset for multiple epochs. During training, the model learned important features related to different dog breeds, and the training loss gradually decreased with each epoch.

```
history = model.fit(
    train_generator,
    validation_data=val_generator,
    epochs=6
)
```

```
Epoch 1/6
43/43 ━━━━━━━━━━━━━━━━━━━━ 360s 8s/step - accuracy: 0.2302 - loss: 21.3027 - val_accuracy: 0.4578 - val_loss: 9.8522
Epoch 2/6
43/43 ━━━━━━━━━━━━━━━━━━━━ 393s 9s/step - accuracy: 0.7625 - loss: 2.9652 - val_accuracy: 0.5904 - val_loss: 7.3552
Epoch 3/6
43/43 ━━━━━━━━━━━━━━━━━━━━ 405s 9s/step - accuracy: 0.9501 - loss: 0.4228 - val_accuracy: 0.5994 - val_loss: 6.3206
```

Outcome:

The model was successfully trained and achieved good accuracy.

Activity 7: Saving the Model

After training, I saved the trained model in .h5 format. This file stores the model architecture, trained weights, and optimizer configuration.

```
]: model.save("dog_breed_model.h5")
   print("Model saved successfully")
```

Outcome:

The trained model was saved for future predictions and deployment.

Activity 8: Testing the Model

Finally, I tested the trained model using unseen images. I provided an input image to the model, and it predicted the corresponding dog breed by mapping the predicted class index to the breed name.

```
pred = model.predict(img_array)

class_names = list(train_generator.class_indices.keys())
predicted_class = class_names[np.argmax(pred)]

print("✅ Predicted Dog Breed:", predicted_class)
```

```
1/1 ━━━━━━━━━━━━━━━━━━━━ 1s 1s/step
✅ Predicted Dog Breed: pekinese
```

Outcome:

The model successfully predicted the dog breed from the input image.

Application Building

I developed a web-based application using Flask to deploy the trained dog breed classification model. The main goal of this milestone is to allow users to upload a dog image through a browser and receive the predicted dog breed as output.

Activity 1: Designing the User Interface using HTML, CSS, and JavaScript

I created the front-end part of the application using HTML. I designed simple and user-friendly web pages so that users can easily interact with the system.

I created the following HTML pages:

- index.html – Displays the home page and basic introduction of the project

```html
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <title>Dog Breed Prediction</title>
    <link rel="stylesheet" href="{{ url_for('static', filename='css/style.css') }}">
</head>
<body>

<div class="hero">
    <div class="hero-content">
        <h1>🐶 Dog Breed Prediction</h1>
        <p>Upload a dog image and instantly predict its breed using AI</p>
        <a href="/predict" class="btn">Start Prediction</a>
    </div>
</div>

</body>
</html>
```

- predict.html (or upload.html) – Allows the user to upload a dog image

```html
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <title>Upload Image</title>
    <link rel="stylesheet" href="{{ url_for('static', filename='css/style.css') }}">
</head>
<body>

<div class="card">
    <h2>Upload Dog Image</h2>

    <form method="POST" enctype="multipart/form-data">
        <input type="file" name="image" required>
        <button type="submit" class="btn">Predict Breed</button>
    </form>

    <a href="/" class="link">⬅ Back to Home</a>
</div>

</body>
</html>
```

- output.html – Displays the predicted dog breed result

```html

<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <title>Prediction Result</title>
    <link rel="stylesheet" href="{{ url_for('static', filename='css/style.css') }}">
</head>
<body>

<div class="card">
    <h2>Prediction Result</h2>

    <img src="/{{ image_path }}" class="result-img">

    <h3>Predicted Breed:</h3>
    <p class="breed">{{ breed }}</p>

    <a href="/predict" class="btn">Try Another Image</a>
</div>

</body>
</html>
```

To improve the appearance and usability of the web pages, I used CSS for styling and JavaScript for basic interactivity. This helped in making the application visually clear and easy to use.

Activity 2: Building the Backend using Flask (Python)

I developed the backend logic of the application using Flask, which is a lightweight Python web framework.

First, I imported the required libraries such as:

- Flask – to create the web application
- TensorFlow/Keras – to load the trained model
- OS – to handle file paths and image uploads

Then, I created a Flask application object and loaded the trained CNN model (.h5 file) using the load_model() function. This model is used to predict the dog breed from the uploaded image.

app.py > ...

```python

from flask import Flask, render_template, request
import os
from werkzeug.utils import secure_filename

app = Flask(__name__)

UPLOAD_FOLDER = 'static/uploads'
os.makedirs(UPLOAD_FOLDER, exist_ok=True)
app.config['UPLOAD_FOLDER'] = UPLOAD_FOLDER


@app.route('/')
def index():
    return render_template('index.html')


@app.route('/predict', methods=['GET', 'POST'])
def predict():
    if request.method == 'POST':
        file = request.files['image']
        if file:
            filename = secure_filename(file.filename)
            filepath = os.path.join(app.config['UPLOAD_FOLDER'], filename)
            file.save(filepath)

```

Activity 3: Routing and Connecting Frontend with Backend

I implemented URL routing to connect HTML pages with Python functions.

- The root URL (/) is mapped to the home page
- When the user uploads an image and clicks the Predict button, the request is sent to the Flask backend using the POST method
- Flask receives the image file and processes it

This routing mechanism allows smooth communication between the user interface and the backend logic.

```
@app.route('/predict', methods=['GET', 'POST'])
def predict():
    if request.method == 'POST':
        file = request.files['image']
        if file:
            filename = secure_filename(file.filename)
            filepath = os.path.join(app.config['UPLOAD_FOLDER'], filename)
            file.save(filepath)
```

Activity 4: Image Upload and Preprocessing

I handled the uploaded image file from the HTML page.

- The uploaded image is saved in a local folder
- The image is resized and preprocessed to match the input shape required by the CNN model
- Necessary normalization and formatting are applied

This preprocessing step ensures that the image is suitable for prediction by the trained model.

Activity 5: Running and Testing the Application

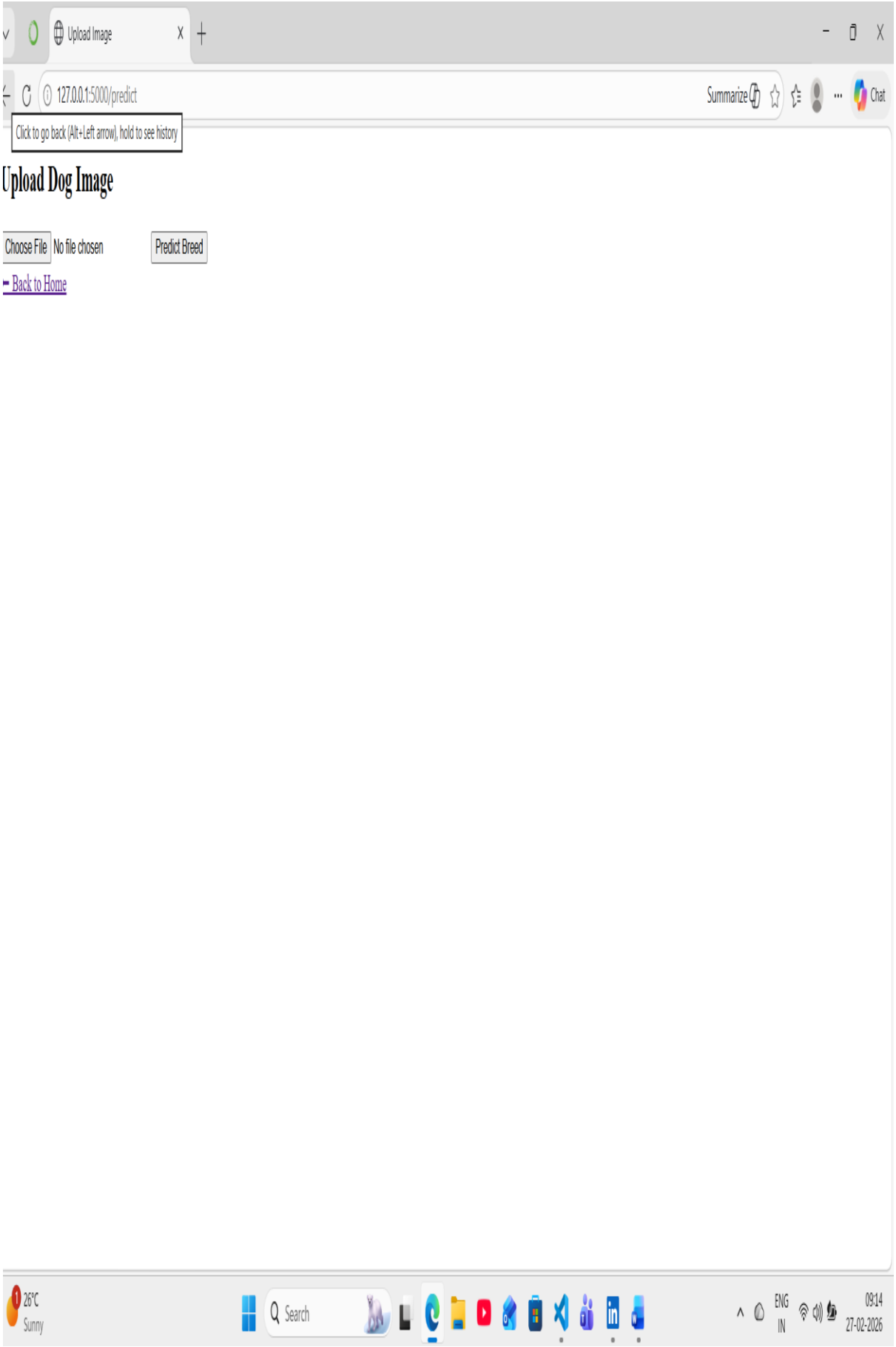
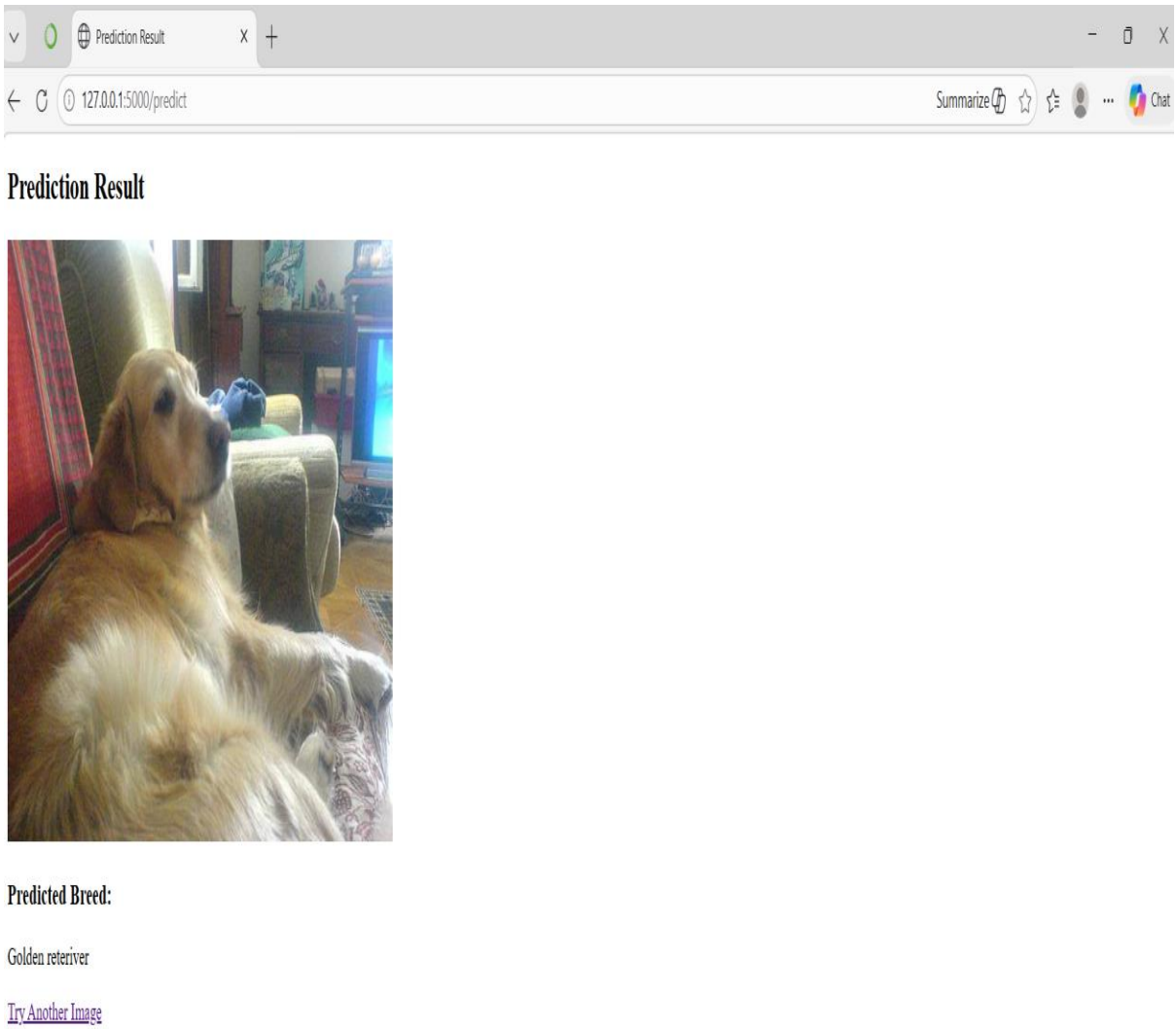
I executed the Flask application locally.

- The application is run using the command python app.py
- The web app runs on http://127.0.0.1:5000/
- I tested the application by uploading different dog images
- The system successfully predicted the dog breeds

This confirms that the application works correctly from image upload to result display.

```
PS C:\Users\DELL\Downloads\DOG_BREED_PREDICTION> & C:/Users/DELL/anaconda3/anaconda/Anacondaaa/envs/dogbreed/python.exe c:/Users/DELL/Downloads/DOG_
BREED_PREDICTION/app.py
 * Serving Flask app 'app'
 * Debug mode: on
WARNING: This is a development server. Do not use it in a production deployment. Use a production WSGI server instead.
 * Running on http://127.0.0.1:5000
Press CTRL+C to quit
 * Restarting with stat
 * Debugger is active!
 * Debugger PIN: 126-371-244
```

After successfully building the Flask web application, I tested the system using multiple dog images. The application was able to correctly accept the input image from the user interface, preprocess it, and pass it to the trained CNN model.


Upload Image
127.0.0.1:5000/predict
Summarize
Chat
Click to go back (Alt+Left arrow), hold to see history
Upload Dog Image
Choose File
No file chosen
Predict Breed
Back to Home
26°C
Sunny
Search
ENG
IN
09:14
27-02-2026

The model successfully predicted the correct dog breed for the uploaded images, and the predicted breed name was accurately displayed on the output web page. This confirms that the integration between the front-end interface, Flask backend, and the trained deep learning model worked properly.